Quantum Chess Engine: Project Implementation Report

Prepared for: Faculty review and team revision

Project: Quantum Chess Engine

Course: Quantum Computing & Algorithms

Report scope: Current repository implementation, configuration, circuit designs, system comparison, and known limits

Benchmark run: 14 October 2026, four seeded 3×3 positions

Executive summary

This project is an educational chess application that connects a classical chess engine to Qiskit circuit simulation. It contains three demonstrations with different goals:

1. **8×8 hybrid game:** classical code generates legal moves and scores them. A Grover circuit samples indices of the classically identified best-scoring moves. Minimax is available as a fallback.

2. **3×3 pure-circuit demonstration:** a supplied legal move is encoded using coordinate and square-status registers, passed through a small reversible-gate prototype, and measured. The actual game transition is still applied by classical code.

3. **2×2 black-pawn demonstration:** a minimal board makes coordinate encoding and controlled gates easier to inspect. It illustrates circuit mechanics rather than complete chess rules.

The project does not represent the complete board or full chess game tree coherently in qubits. The hybrid Grover path receives candidate moves and target indices prepared classically. The pure-circuit direction and status transformation are incomplete prototypes. The current benchmark reports execution time, sampled extra peak process RAM, legal-move count, qubits, gates, and circuit depth. It is a simulator resource comparison, not evidence of quantum advantage.

1. Project purpose and scope

The goal is to make the relationship between a chess application and quantum-computing concepts explainable. Classical modules handle board state, move legality, and game transitions. Quantum modules construct circuits and simulate gates. Small boards reduce the number of registers and make gate wiring easier to discuss.

The project is best presented as a **classical chess engine with quantum circuit demonstrations and a hybrid Grover experiment**. “Pure quantum” is the name used for a small-board circuit route; it does not mean that the running game state is quantum or that all move rules execute in a quantum processor.

2. Overall architecture

```
main.py / UI
  ├── 8×8 game ──> Game + GameState (classical source of truth)
  │             └── MoveGenerator + Rules (classical legal candidates)
  │               └── HeuristicEvaluator (classical scores)
  │                 └── QuantumMoveSearcher / GroverSearch
  │                   └── Aer simulation, measurement, index decode
  ├── 3×3 game ──> PureQuantumCircuitBuilder ──> Aer simulation
  │             └── Game.make_move applies the chosen move classically
  └── 2×2 demo ──> same small-board circuit concepts with one Black Pawn
```

The main software layers are:

- **Presentation and orchestration:** main.py, ui/gui.py, ui/renderer.py, and ui/animations.py provide the menu, game interaction, board rendering, and move animation.
- **Chess domain:** engine/ defines positions, pieces, board storage, game state, moves, rules, move generation, and evaluation.
- **AI:** ai/ contains heuristic scoring, minimax, the 3×3 classical selector, hybrid orchestration, and quantum move-search integration.
- **General quantum search:** quantum/ contains encoding/decoding, oracle and diffuser, Grover construction, measurement handling, and the Aer simulator wrapper.
- **Small-board pure-circuit prototype:** pure_quantum_engine/ allocates registers, builds the circuit, executes it, and exposes a benchmark helper.
- **Evaluation and supporting materials:** evaluation/, docs/, tools/, and tests/ hold the benchmark, documentation, inspection scripts, and automated checks.

3. Implemented systems

3.1 8×8 hybrid game

The normal 8×8 board lives in a classical GameState. MoveGenerator.generate_legal_moves supplies the legal candidate list. HeuristicEvaluator.score_move computes a score using capture value, pawn advancement, center preference, and piece-specific center preference. The quantum searcher identifies the maximum score and sends all tied best-move indices to Grover. The measured index is mapped back to its Move object and validated against the list. If the quantum path cannot provide a move, HybridEngine has a depth-3 minimax fallback. A position with one legal move bypasses Grover.

The quantum register represents an **index into a classical legal-move list**, not the full 8×8 board. For N candidates, it uses ceil(log2(N)) move-index qubits (at least one) plus one phase-kickback flag. Padding states above the candidate count have no associated move.

3.2 3×3 pure-circuit demonstration

The default 3×3 layout is a White Knight at (2,1) and a Black Pawn at (0,0). The user or UI supplies a move. The circuit builder encodes source and target coordinates, looks up square statuses from the classical board at circuit-construction time, adds limited direction logic and a status-operation attempt, then measures the registers. The wrapper prints circuit output and metrics. The Game.make_move

implementation then performs the actual board transition classically. The 3×3 demonstration's AI is configured with HybridEngine(use_quantum=False), so this mode does not use Grover for its AI turn.

3.3 2×2 black-pawn demonstration

The default 2×2 layout contains one Black Pawn at (0,0). The coordinate convention is column bits followed by row bits. For example, (row=1, col=0) becomes column 00 + row 01, or 0001. The compact small-board status is three bits: empty 000, Black Pawn 001. The demo makes it easier to follow basis-state initialization, zero-control normalization, multi-controlled gates, and measurements.

3.4 Classical 3×3 comparison system

The benchmark's classical selector uses depth-3 minimax on the same four 3×3 positions. This is the classical baseline. It reports runtime and extra peak RAM; quantum resource metrics are N/A because it constructs no circuit.

4. Project files and responsibility map

Path	Responsibility
main.py	Interactive menu and connections between game/demo modes
config.py	Board defaults, demo layouts, shot count, backend/noise settings, UI constants
engine/board.py, state.py, game.py	Board storage, current side/history, and classical state transition
engine/move_generator.py, rules.py	Piece movement generation and legal-move filtering
engine/piece.py, pieces.py, position.py, move.py, constants.py	Chess data model and shared definitions
engine/evaluator.py	Position evaluation and terminal/draw-related logic
ai/heuristic.py	Classical position and per-move scoring
ai/minimax.py	Classical minimax search with alpha-beta pruning and move ordering
ai/hybrid_engine.py	Hybrid AI orchestration and fallback behavior
ai/quantum_search.py	Classical best-index preparation, Grover invocation, and result fallback
ai/classical_3x3.py	3×3 depth-checked classical selector used for comparison
quantum/encoder.py, decoder.py, registers.py	Conversion between classical data, registers, and measured values
quantum/oracle.py, diffuser.py, grover.py	Grover circuit parts and search execution
quantum/measurement.py, simulator.py	Shot-result processing and Qiskit Aer

Path	Responsibility
	configuration/execution
quantum/circuit_builder.py	Reusable coordinate/direction/status circuit construction helpers
pure_quantum_engine/registers.py	Source/target, direction, status, ancilla, and measurement registers
pure_quantum_engine/circuit.py	Small-board circuit assembly and gate-level logic
pure_quantum_engine/engine.py	Circuit execution and returned measurements/metrics
pure_quantum_engine/benchmark.py	Standalone pure-circuit benchmark helper
ui/gui.py, renderer.py, animations.py	User input, display, and move animation
evaluation/compare_3x3_systems.py	Four-position measurement, CSV/graph/report generation
evaluation/best_move.py	Compatibility entry point for the comparison script
tools/inspect_circuit.py, inspect_pure_circuit.py	Circuit inspection and saved diagram generation
tests/	Unit and integration checks for board, rules, encoding, Grover, and pure circuit

5. Configuration and execution settings

Setting	Current value	Meaning
BOARD_SIZE	8	Default board side length; demo routes create smaller boards explicitly
INITIAL_PIECES_3x3	White Knight (2,1); Black Pawn (0,0)	Small-board game/circuit example
INITIAL_PIECES_2x2	Black Pawn (0,0)	Minimal gate-level example
QUANTUM_SHOTS	1024	Default shot count in the shared simulator configuration
QUANTUM_BACKEND	aer_simulator	Local Qiskit Aer simulator identifier
QUANTUM_NOISE_ENABLED	True	Shared wrapper's default noise setting
QUANTUM_NOISE_PROB	0.01	Illustrative depolarizing probability; selected two-qubit gate rate is configured at ten times this value
UI	60 FPS, 600×680 window	Default Pygame dimensions and target frame rate

The 3×3 comparison uses separate settings for reproducibility and practicality: Aer CPU matrix_product_state, noise disabled, one shot for each pure-circuit case, and 2048 shots for the hybrid Grover case. The benchmark's four positions are built with seed 20261013. It excludes simulator setup from per-system timing. These benchmark values should not be conflated with the shared application defaults above.

6. Qubit encoding and register allocation

A register of n qubits represents 2^n computational basis states. Each qubit state can be written $\alpha|0\rangle + \beta|1\rangle$, with $|\alpha|^2 + |\beta|^2 = 1$. Measurement returns a classical value with probabilities determined by squared amplitudes.

Small boards

A 2×2 or 3×3 coordinate uses two bits per axis, so each square coordinate takes four qubits. A 3×3 board therefore has nine valid squares among the sixteen basis encodings available to four bits. A small-board pure circuit allocates:

Register	Qubits	Role
Source coordinate	4	Source column and row
Target coordinate	4	Target column and row
Direction	4	Direction/movement workspace
Source status	3	Encoded source-square status
Destination status	3	Encoded destination-square status
Ancilla	5	Temporary computation/workspace
Total	**23**	Plus 23 classical measurement bits

The small-board status uses a color bit and compact piece-category bits. Some piece categories share codes, so it does not uniquely represent all six standard piece types.

8×8 allocation distinction

The separate pure-circuit register allocator would use six bits per square coordinate, twelve for source and target together, four for direction, six each for source/destination status, and five ancillas: 33 qubits total. This is an allocation formula, not proof of a complete 8×8 quantum rules circuit. Normal 8×8 hybrid move selection uses the much smaller move-index register described above.

7. Gate-level explanation

Gate	State/action	Use in this project
X	Swaps zero and one basis states	Loads encoded 1-bits; flips controls for equality tests; prepares a Grover flag and diffuser portions
H	Maps zero to an equal plus state; maps one to an equal minus state	Creates a uniform move-index superposition; prepares the minus phase-kickback flag
CX	Flips target if one control is 1	Conditional reversible bit operation used in logic patterns
CCX	Flips target if both controls are 1	Toffoli gate for reversible Boolean conditions
MCX	Flips target if every control is 1	Coordinate/status lookup predicates and Grover index oracle
Z / MCZ	Adds a negative-one phase to selected one-basis states	Phase marking and diffuser construction
Measurement	Samples and collapses qubits into classical bits	Returns Grover index samples or pure-circuit diagnostic strings

In Grover search, the flag is prepared in $|-\rangle$ by X then H. Since X acting on $|-\rangle$ returns $-|-\rangle$, a controlled flip of the flag changes the sign of the matching index amplitude. This is phase kickback. A phase change alone does not change an outcome probability; the diffuser interferes amplitudes and increases or decreases the chance of later measurement outcomes. The diffuser implements reflection about the mean, $D = 2|s\rangle\langle s| - I$.

In the pure-circuit status extractor, the builder iterates over known occupied squares in Python, flips zero-valued coordinate controls temporarily, and applies MCX gates to toggle status bits for a coordinate match. This is a circuit-generation-time lookup of a classical board. It is not a quantum memory holding a superposition of all board squares.

8. Move-selection pipelines

Hybrid Grover path

1. The classical move generator returns legal moves.
2. The classical heuristic scores every legal move.
3. The searcher identifies all indices tied for maximum score.
4. Grover initializes a uniform superposition over the padded index space, 2^n.
5. The oracle phase-marks target indices with controlled gates and phase kickback.

6. The diffuser reflects amplitudes about their mean. The oracle/diffuser pair repeats for the iteration count used by the implementation.

7. Aer samples the move-index register. The result decoder maps bits to an integer and then to the original Move.

8. Invalid/padded/non-target samples use a deterministic best-index fallback. The engine can use minimax if the quantum path returns no move.

The ideal Grover probability analysis assumes a useful oracle with a known query cost. Here, classical legality and heuristic target selection are already complete before the Grover circuit runs. Describe this as amplification of classically prepared best-move indices.

Pure-circuit path

1. A user/UI-selected move supplies source and target squares.

2. The builder initializes coordinate and status registers.

3. The circuit appends status extraction, limited direction operations, and a status-operation attempt.

4. The simulator executes the circuit and measures registers.

5. The classical game applies the supplied move to the authoritative GameState.

The measured pure-circuit bit string is diagnostic output. The code does not decode a complete board state from it.

9. Functional implementation phases

The implementation can be explained in six functional phases. These describe dependency order and project organization, not a dated release chronology.

1. **Classical chess foundation:** board, piece, position, move, game state, movement rules, move generation, and state transitions.

2. **Classical AI baseline:** evaluation, heuristic move scoring, minimax, alpha-beta pruning, and 3×3 minimax selector.

3. **Quantum representation:** coordinate/status encoders, register allocation, classical-to-basis initialization, decoder, and measurement handling.

4. **Quantum search and simulation:** Grover index oracle, diffuser, Aer execution, sampling, index decoding, and fallback behavior.

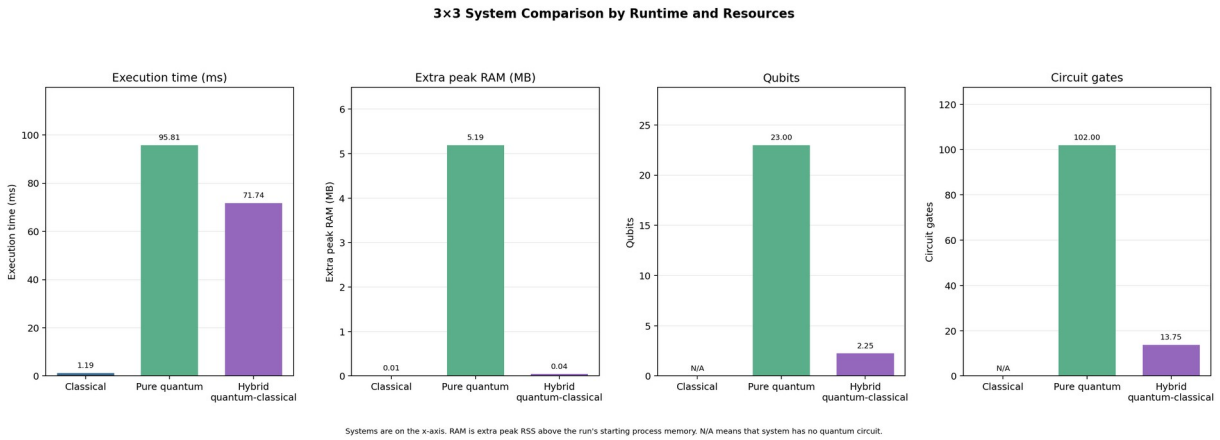
5. **Demonstration integration:** interactive menu, 8×8 hybrid game, 3×3 pure-circuit path, 2×2 gate walkthrough, circuit diagrams, and UI integration.

6. **Evaluation and faculty material:** fixed-position 3×3 benchmark, simple resource metrics, CSV/plot/report outputs, architecture documentation, Q&A, and folder guide.

10. Evaluation design and measured results

The comparison uses the same four seeded 3×3 positions for each system. The mean legal-move count is 2.25. The systems perform different operations: the classical system selects a move with depth-3 minimax; the pure quantum system builds and simulates the 23-qubit circuit for a supplied legal move; the hybrid system prepares heuristic-best indices classically and runs the Grover move-index circuit.

System	Avg. time (ms)	Extra peak RAM (MB)	Avg. legal moves	Qubits	Gates	Circuit depth
3×3 Classical	1.19	0.0068	2.25	N/A	N/A	N/A
3×3 Pure Quantum	95.81	5.1924	2.25	23.00	102.00	32.25
3×3 Hybrid Quantum-Classical	71.74	0.0430	2.25	2.25 avg.	13.75 avg.	10.25 avg.



Four metric panels with systems on the x-axis and time, RAM, qubits, and gates as y-axis metrics

- **Execution time:** measured operation time per position. Classical timing covers move selection; pure timing covers circuit construction and simulation; hybrid timing covers candidate scoring, Grover circuit construction, transpilation, and simulation. Simulator setup is excluded.
- **Extra peak RAM:** estimated increase in resident memory above the pre-step process baseline. A sampler checks every 2 ms, so short peaks can be missed. Small values can round to 0.00 MB in the graph.
- **Average legal moves:** gives context about candidate-list size.
- **Qubits, gates, depth:** circuit structural metrics. Classical values are N/A because no circuit exists.

These are local simulator measurements. They depend on the host, simulator version, transpilation, and measurement settings. They do not compare execution on a real quantum processor. The operations are not identical end-to-end tasks, so runtime is useful as a basic resource indicator but not a direct algorithmic speedup comparison. The benchmark does not assess move quality or win rate.

11. Result interpretation

In this run, depth-3 classical selection had the lowest measured runtime and extra memory. The pure-circuit simulator path cost more time and memory because it constructs and executes a 23-qubit prototype circuit. The hybrid Grover circuit used fewer qubits and gates than the pure-circuit path, but it still includes classical candidate scoring and simulator work. Its measurements do not show a quantum advantage. On classical hardware, statevector simulation can become expensive as qubit count grows because it stores amplitudes for exponentially many basis states.

12. Current limitations and next engineering steps

- **Classically prepared oracle:** legality and target scoring occur before Grover. A useful quantum-search claim needs a coherent predicate and complete accounting of oracle construction/evaluation cost.
- **Incomplete pure-circuit movement logic:** direction detection consists of limited gate patterns, not a general reversible coordinate subtractor/comparator.
- **Incomplete board transition:** the circuit lacks a quantum register for the whole board and does not implement every move detail such as source clearing, captures, castling, en passant, promotion, or self-check handling.
- **Encoding ambiguity:** compact small-board codes reuse types. The larger sparse mapping also needs careful collision/empty-piece handling before using it as a complete representation.
- **Classical authority:** classical game code applies the actual state update after circuit simulation.
- **Benchmark boundaries:** only four tiny positions are used; no quality, scaling, hardware, or noise-on comparison is included.
- **Chess rule coverage:** special move and tournament-standard behavior should be checked against an independent rules reference before making a full-compliance claim.

A practical next sequence is to define an injective encoding including empty and every colored piece, specify a reversible move predicate and full reversible state update, verify gates against basis-state truth tables, measure Grover success probability against the amplitude formula on controlled cases, then run a fair scaling study that includes all classical and oracle costs.

13. Run and inspect the project

From the repository root:

```
python main.py
```

Use the menu to open the 8×8 hybrid game, 3×3 pure-circuit demonstration, circuit diagrams, or the 2×2 black-pawn examples. For the simple system comparison:

```
python evaluation/compare_3x3_systems.py
```

The script writes the summary CSV, per-position CSV, graph, and generated comparison report under evaluation/results/ and docs/reports/. Circuit inspection helpers:

```
python tools/inspect_circuit.py
python tools/inspect_pure_circuit.py
```

Install project dependencies from requirements.txt in the chosen Python environment if they are not present. The app uses Pygame for the graphical interface and Qiskit/Aer for quantum simulation.

14. Existing Markdown documentation covered by this report

Markdown document	Main purpose and where this report uses it
README.md	Project overview, installation, usage, modes, and quick commands
ARCHITECTURE.md	Top-level architecture and dependency/data-flow reference
JOURNAL.md	Development notes and implementation history
docs/architecture.md	Layered module architecture and application data flows
docs/circuit_design.md	Circuit register/gate design detail
docs/quantum_design.md	Quantum design intent, Grover, and encoding concepts
docs/2x2_qc_architecture.md	2×2 scenario architecture and coordinate/gate explanation
docs/2x2_pure_quantum_grover.md	Combined 2×2 Grover and pure-circuit walkthrough
docs/reports/chess_engine_quantum.md	Detailed system architecture, gates, encodings, hybrid path, and limitations
docs/reports/ques_ans.md	Faculty oral-exam questions and answers, including challenging prompts
docs/reports/folder_dir.md	File-by-file repository and artifact map
docs/reports/3x3_system_comparison.md	Current benchmark definitions, caveats, and detailed comparison
docs/reports/project_implementation_report.md	This combined faculty and team revision report

The Q&A bank is a companion revision tool: it includes questions on system scope, qubit allocation, gate operation, phase kickback, Grover iteration, circuit limitations, configuration, simulator constraints, and defensible claims about quantum advantage.

15. How to present the project accurately

A concise faculty explanation is: “Our project keeps the chess board and legal move generation classical. In 8×8 hybrid play, we score legal moves classically, then demonstrate Grover amplitude amplification over the indices of the best-scoring candidates. Separately, our 3×3 and 2×2 modes build and simulate small coordinate/status circuits to teach reversible gate behavior. The measured circuit outputs do not update a complete quantum board. Our 3×3 comparison records simple simulator runtime, sampled extra RAM, and circuit resources; the current results are educational measurements and do not establish quantum speedup.”

Appendix A. Quick oral-revision prompts

1. What state does GameState store, and which module applies the actual move?
2. What exactly is in superposition in the hybrid Grover circuit?
3. Why does a phase oracle need a diffuser before measurement probabilities change?
4. Why is the 2×2 board useful for explaining coordinate-controlled MCX operations?
5. Why does a 3×3 square coordinate use four qubits?
6. What parts of the pure-circuit path remain classical?
7. What does the benchmark’s extra peak RAM estimate, and what can its sampler miss?
8. Why do classical rows report circuit metrics as N/A?
9. What evidence would be needed to support an advantage claim?
10. Which implementation step would make the pure circuit a complete reversible board transition?